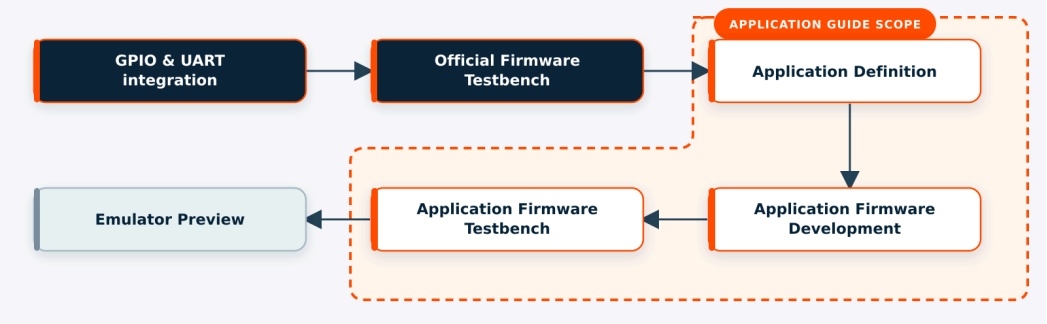
ChampionCHIP eXperience

Phase 3 Challenge - Design Submission Guide

This document defines the mandatory deliverables regarding the architecture, implementation, and functional validation of your project. Participants must present the architecture, implementation, validation, and results of the integration of their RISC-V SoC with the virtual environment, including technical documentation and a functional demonstration of the processor interacting with the emulator.

1. Project Overview

Describe the project's overall goal, the architecture used, and the system's general operation. Focus on explaining your proposed application and how the Application Firmware uses the UART and GPIO interfaces to interact with a real-world scenario. The "Virtual Environment / Device Emulator" concept is merely to illustrate how the RISC-V processor reacts to external stimuli. **Your team's focus must be strictly on developing the application logic and the firmware (AWS FPGA direct implementation will not be covered in this part). The image below illustrates the project's workflow for this phase:**



2. GPIO & UART Integration (System Architecture)

Present the system architecture focusing on peripheral connections.

- **Integration:** Explain the design developed for the practical use and integration of the GPIO and UART in the project.
- **Block Diagram:** It is mandatory to include in the report the block diagram generated directly in **ChipInventor** to illustrate the hardware connections. (Suggestion: use colored wires and represent the project as a block).

3. Official Firmware Testbench

To prove the correct operation of the GPIO and UART, the **Official Firmware Testbench** must be used:

- **Objective:** Run the official testbench to validate the GPIO and UART integration, ensuring the correct communication of the base architecture before starting the application development.
- **Requirements:** Execute the test on the integrated design and include execution logs or waveform images in the report. This proves the successful functioning of the UART transmission/reception and GPIO triggering.
- **GitHub:** https://github.com/championchip-experience-community/CCX_Malaysia_Edition_Stage_3/tree/main/RVBL-GPIO-UART-Test-Firmware

4. Application Definition

Clear explanation of the **application's goal and project structure**. The following key points must be clearly defined:

- **System Context:** What system will use the processor?
- **System Input:** What information enters the system?
- **System Processing:** What does the firmware calculate or decide?
- **System Output:** What result or action does the processor produce?
- **Emulator Values (`./emu`):** Understand how the emulator works for the specific use of your project.
- **GitHub Emu:** https://github.com/championchip-experience-community/CCX_Malaysia_Edition_Stage_3/tree/main/RVBL-Emulator

5. Firmware Development and Processing

- Describe the logic of the **participants' Firmware** in detail.
- Present an in-depth explanation of the data processing (e.g., how data received via UART is handled, how the CRC/Zmmul calculation is performed, and how control signals are sent to the GPIOs).
- **GitHub:** https://github.com/championchip-experience-community/CCX_Malaysia_Edition_Stage_3/tree/main/RVBL-Firmware-Builder

6. Custom Firmware Validation and Testbench

The proof of operation must be supported by the team's tests:

- **Team's Testbench:** Present and explain the custom testbench (created by the team) used to validate the architecture.
- **Tested Firmware:** Demonstrate, through logs or waveforms, that the developed firmware was properly tested and validated via the team's testbench prior to the final execution.

7. Video Demo

Produce a presentation video with a **maximum duration of 8 minutes**. The video must be dynamic, focused on presenting and "selling" the idea, and strictly structured into the following topics:

1. **Technical Part:** Detail the architecture, hardware design, and firmware implementation.
2. **Application Logic:** Demonstrate the working application, clearly showing the connection logic between the emulator (`./emu`) and the RISC-V processor.
3. **Pitch (Selling the idea):** Use a professional posture to sell the project concept, highlighting its value and practical functionality.

8. Report Format

The report must contain topics 1 to 5 of this document in PDF format, with a **maximum of 7 pages** and a maximum file size of 10 MB.

9. Expected Files

The project submission must contain the following mandatory elements:

1. **Technical Report (PDF file, max. 7 pages).**
2. **GitHub Link**
3. **Video Demo (YouTube link, as instructed on the platform).**
4. **Project Files:**
    - **All developed source codes (C, Verilog, etc.).**
    - **Complete firmware, which must be delivered in both binary and assembly formats.**

10. Submission Deadline

All required deliverables, including the PDF report, source codes, and the video demonstration link, must be submitted through the official ChampionCHIP platform no later than **September 30, 2026** at **23:59 MYT.**

Late submissions will not be accepted, as per the general competition rules.

Note 1 - Submission Integrity:

All submitted code (C files, firmware, and RTL codes) must represent the team's own work. Copying, sharing, or reusing code from other participating teams is strictly prohibited. Submitting third-party or externally sourced implementations as the team's own work is also prohibited. Evaluators may reproduce the simulation and execution flow on AWS using the submitted files, and evidence of unauthorized code sharing or copying may result in penalties or disqualification.


ChampionCHIP
eXperience

Table of Contents

1. PIN Controller (GPIO)..... 4
1.1 Memory Map..... 4
1.2 Peripheral Registers..... 4
1.3 Functional Description..... 6
2. Serial Controller (UART)..... 8
2.1 Memory Map..... 8
2.2 Peripheral Registers..... 8
2.3 Functional Description..... 10

1. PIN Controller (GPIO)

This section covers the operation of the pin controller built into the IP, which allows control of the pins_io bus (referred to as GPIO pins). This peripheral has the following characteristics:

- Ability to control up to 32 pins (only 8 are used).
- Registers for controlling the direction, reading, and writing to bidirectional pins.
- Basic protection against bus contention (logic level conflicts).

The peripheral does not support current drive or internal pull-up resistors.

1.1 Memory Map

The GPIO peripheral has 3 control registers mapped to memory, which can be accessed by the user starting at base address 0xF0000000, as shown in Table X. The memory map for the registers, containing the offsets for access, is shown in Table X:

Table 1. IP Feature Set

Offset	Register	Description
0x0000	DATAOUT	Allows writing a logic level to the output pins.
0x0004	DATAIN	Allows reading the logic level of the input pins.
0x0008	DATADIR	Controls which pins are input or output.

These registers must be accessed with 32-bit aligned addresses (the two lower bits of the address are ignored).

1.2 Peripheral Registers

The peripheral is characterized by the control of 8 bidirectional pins on the **pins_io** bus, which can act as either input or output, using the registers shown in Table X. Each bit of the registers references an input/output pin.

The following is a description of each register and its role in controlling the pins:

1.2.1 DATAOUT Register

This register configures the logic levels to be driven on each GPIO pin, provided they are configured as outputs in the DATADIR register.

Table 2. IP Feature Set

DATAOUT Register			
Register Offset		0x0000	
Bits	**Name**	**Attr.**	**Description**
[31:8]	Reserved	-	-
[7:0]	PIN_VALUE	RW	Logic level for the output pins.

1.2.2 DATAIN Register

This register reflects the logic levels applied to the GPIO pins, provided they are configured as inputs in the DATADIR register.

Table 3. IP Feature Set

DATAIN Register			
Register Offset		0x0004	
Bits	**Name**	**Attr.**	**Description**
[31:8]	Reserved	-	-
[7:0]	PIN_VALUE	R	Logic level for the input pins.

1.2.3 DATADIR Register

This register configures the pins as either input or output.

Table 4. IP Feature Set

DATADIR Register			
Register Offset		0x0008	
Bits	**Name**	**Attr.**	**Description**
[31:8]	Reserved	-	-
[7:0]	PIN_DIR	RW	Pin direction, where 0 is for input and 1 is for output.

CONFIDENCIAL

1.3 Functional Description

The following is the functional description of the peripheral. Each bit of the registers references an input/output pin. For example, bit 6 of the DATADIR register determines whether pin P6 (i.e., bit 6 of the **pins_io** port) is an input (if it has a value of 0) or an output (if it has a value of 1).

1.3.1 Independent Pin Control

Each pin has its own dedicated hardware to manage it based on the respective bit in the control registers. Figure X shows the high-level diagram of the peripheral, illustrating its main internal components and their interconnections:

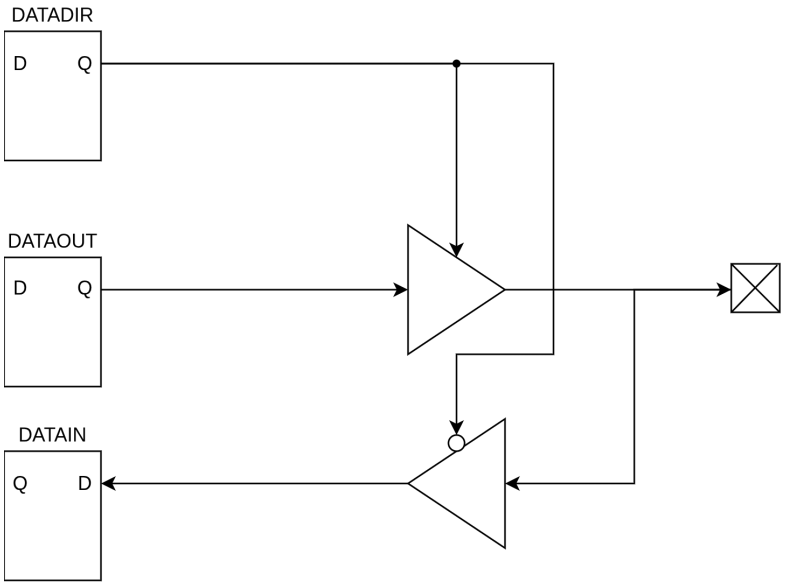


Figure 1. I/O Pin Control Structure

The figure illustrates the control logic circuit for a single GPIO pin. As shown, pin control is performed by a corresponding bit—represented by a flip-flop—within each of the three peripheral registers: **DATADIR, DATAOUT, and DATAIN**.

Bit X of the DATADIR register acts as the direction selector for the pin. It determines whether **pin X** will function as an output (reflecting the logic level of DATAOUT) or as an input (where the external level applied to the pin defines the value read into DATAIN).

To prevent signal contention, the circuit employs buffers with complementary enable signals:

- **Output Mode** (DATADIR[X] = 1): The output buffer is enabled, driving the DATAOUT value onto the pin. Simultaneously, the input buffer is disabled (isolated), preventing this signal from affecting the DATAIN register.
- **Input Mode** (DATADIR[X] = 0): The output buffer is disabled, placing the output in a high-impedance state. The input buffer is enabled, allowing the external voltage on the pin to be sampled by DATAIN.

Since the peripheral features eight I/O pins, this base hardware is replicated eight times (out of a maximum of 32).

1.3.2 Example Assembly Code

Below is an example Assembly code illustrating the interface between the RISC-V core (which executes the code) and the GPIO peripheral (which provides specialized functionality) using memory-mapped registers and load/store instructions:

```
1   .equ GPIO_BASE,       0xF0000000
2   .equ GPIO_DATAOUT,    0x0000
3   .equ GPIO_DATAIN,     0x0004
4   .equ GPIO_DATADIR,    0x0008
5
6   _start:
7       la s0, GPIO_BASE
8
9       /* Set P0 as input and P1 as output */
10      lw t0, GPIO_DATADIR(s0)
11      ori t0, t0, 0x2
12      sw t0, GPIO_DATADIR(s0)
13
14  _loop:
15      /* Read P0 */
16      lw t0, GPIO_DATAIN(s0)
17
18      /* Write to P1 */
19      sw t0, GPIO_DATAOUT(s0)
20
21      j _loop
```

Using the base address (in **s0**) and the **lw** and **sw** instructions, along with the register offsets, the DATADIR register is first loaded with the value 2 (or 0b10 in binary), thereby configuring pin P0 as an input and pin P1 as an output.

Afterward, the DATAIN register is read (using **lw**), and its value is fed to DATAOUT (using **sw**) within a loop.

2. Serial Controller (UART)

This section covers the operation of the embedded peripheral serial controller within the IP, which utilizes the UART protocol to control the **tx_o** and **rx_i** pins. This peripheral features:

- 8-N-1 format: 8 data bits, no parity bit, and 1 start bit.
- Full-duplex transmission with two 8-bit shift registers (FIFOs).
- Registers for control, data transmission, and reception.
- No oversampling.

The peripheral does not support hardware flow control or synchronous serial data transfers (as provided by the USART protocol).

2.1 Memory Map

The UART peripheral features 3 memory-mapped control registers, accessible by the user starting from the base address 0xF1000000, as shown in Table X. The memory map for the registers, including access offsets, is presented below:

Table 5. IP Feature Set

Offset	Register	Description
0x0000	TXDATA	Hold the byte to be sent.
0x0004	RXDATA	Holds the received byte.
0x0008	CONTROL	Controls the operation.

These registers must be accessed with 32-bit aligned addresses (the two lower bits of the address are ignored).

2.2 Peripheral Registers

The peripheral is characterized by the control of serial transmission and reception based on the UART protocol, using the registers shown in Table X.

The following is a description of each register and its role in communication control:

2.2.1 TXDATA Register

This register holds the byte to be transmitted via the **tx_o** pin using the UART protocol, within the DATA field. This byte is only sent upon setting the TRANSMIT bit in the CONTROL register.

Table 6. IP Feature Set

TXDATA Register			
Register Offset		0x0000	
Bits	**Name**	**Attr.**	**Description**
[31:8]	Reserved	-	-
[7:0]	DATA	RW	Data to be transmitted.

2.2.2 RXDATA Register

This register holds the byte received via the **rx_i** pin using the UART protocol, within the DATA field.

Table 7. IP Feature Set

RXDATA Register			
Register Offset		0x0004	
Bits	**Name**	**Attr.**	**Description**
[31:8]	Reserved	-	-
[7:0]	DATA	R	Received data.

2.2.3 CONTROL Register

This register holds the status and control bits of the UART peripheral, which are required for software-driven communication operations. Only three bits are used, each with specific set (activation) and clear (deactivation) conditions:

- **TRANSMIT** control bit: Manually set via software; automatically cleared in the cycle following the write operation.
- **TXDONE** status bit: Set after transmission is complete; cleared after initiating the next transmission.
- **RXDONE** status bit: Set after reception is complete; manually cleared via software.

Table 8. IP Feature Set

CONTROL Register			
Register Offset		0x0008	
Bits	**Name**	**Attr.**	**Description**
[31:3]	Reserved	-	-
2	TXDONE	R[1]	Transmission via **tx_o** pin completed.
1	RXDONE	RW	Data reception via **rx_i** completed.
0	TRANSMIT	W[2]	Start transmission of the byte in TXDATA.

[1] Since the TXDONE bit is written by the transmitter module, software written to this bit will be overwritten in the next clock cycle.
[2] Since the TRANSMIT bit is cleared in the cycle following the write, and a load operation takes at least 4 cycles, the read value will always be 0.

2.3 Functional Description

In addition to the registers shown in Table X, the peripheral features a very simple datapath, consisting of a serial transmission module (**transmitter**) and a serial reception module (**receiver**) that implement the UART protocol for serial communication. The peripheral's operation is based on controlling these two modules using the control registers. Figure X shows the high-level diagram of the peripheral, illustrating its main internal components and their interconnections:

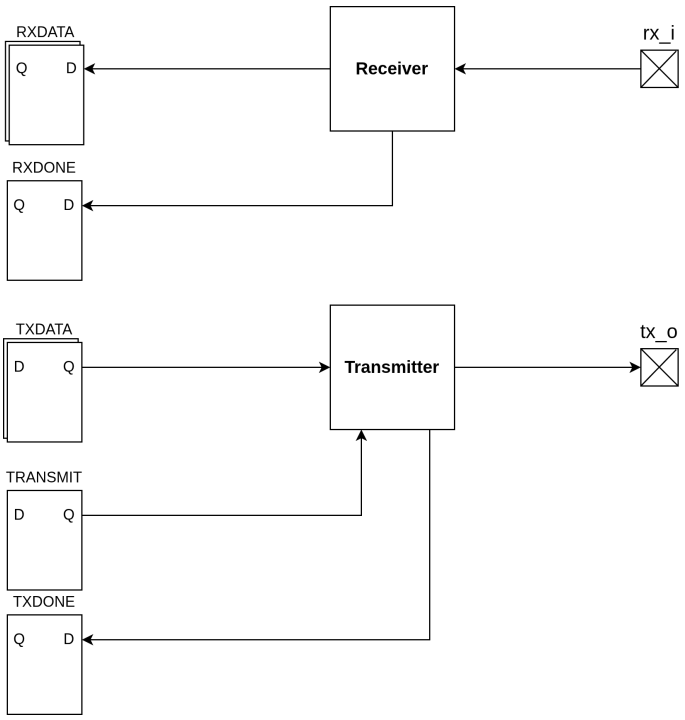


Figure 2. Serial Peripheral Architecture

As shown, the peripheral controls the operation of the transmission and reception modules using the control registers: TXDATA, RXDATA, and CONTROL. It utilizes the TRANSMIT, TXDONE, and RXDONE bits of the CONTROL register, which are implemented as flip-flops.

To enable full-duplex communication (simultaneous transmission and reception), the transmission and reception modules feature independent 8-bit shift registers (FIFOs).

2.3.1 Serial Transmission

The TXDATA register provides the data to be transmitted by the transmitter module via the **tx_o** pin.

Setting the TRANSMIT bit in the CONTROL register loads the value from the DATA field of the TXDATA register into the transmitter's internal shift register ($FIFO_{tx}$) and initiates transmission. Immediately after, the TRANSMIT and TXDONE bits are automatically cleared.

The transmitter then sends the contents of the $FIFO_{tx}$ shift register bit-by-bit over the **tx_o** pin at a baud rate of 115200 bps, starting with the LSB and ending with the MSB, in accordance with the UART protocol for serial transmission.

Upon completion of the transmission, the TXDONE bit in the CONTROL register is set, signaling that new data can be transmitted.

2.3.2 Serial Reception

The RXDATA register holds the data received by the receiver module via the **rx_i** pin.

When data reception begins on the **rx_i** pin, the receiver fills its internal shift register (FIFOrx) bit-by-bit, starting with the LSB and ending with the MSB, following the UART protocol at a baud rate of 115200 bps.

Upon completion of the reception, the RXDONE bit in the CONTROL register is set, signaling that new data is available to be read.

As mentioned previously, the RXDONE bit is only cleared if a zero is manually written to its respective bit in the register. Simply writing zero to the register is sufficient, since the TRANSMIT bit is cleared in the cycle following the write, and the TXDONE bit is updated every clock cycle. The RXDONE bit, however, is only updated when new data is received.

2.3.3 Example Assembly Code

Below is an Assembly code snippet (known as an *echo* program) that illustrates the interface between the RISC-V core (which executes the code) and the UART peripheral (which provides specialized functionality) using memory-mapped registers and load/store instructions:

```
1   .equ UART_BASE,             0xF1000000
2   .equ UART_TXDATA,           0x0000
3   .equ UART_RXDATA,           0x0004
4   .equ UART_CONTROL,          0x0008
5
6   .equ UART_CONTROL_TRANSMIT, 0x01
7   .equ UART_CONTROL_RXDONE,   0x02
8   .equ UART_CONTROL_TXDONE,   0x04
9
10  _start:
11      la s0, UART_BASE
12
13  _check_rxdone:
14      lw t0, UART_CONTROL(s0)
15      andi t0, t0, UART_CONTROL_RXDONE
16      beq t0, zero, _check_rxdone
17      sw zero, UART_CONTROL(s0)
18
19      /* Receive data */
20      lw t1, UART_RXDATA(s0)
21
22  _check_txdone:
23      lw t0, UART_CONTROL(s0)
24      andi t0, t0, UART_CONTROL_TXDONE
25      beq t0, zero, _check_txdone
26
27      /* Send data */
28      sw t1, UART_TXDATA(s0)
29
30      j _check_rxdone
```


CONFIDENCIAL

The intended function is to take the data read from the **rx_i** pin and send it back out through the **tx_o** pin.

Using the base address (in **s0**) and the **lw** and **sw** instructions, along with the register offsets, we must first verify if we can read the received data before actually reading it.

Thus, the value of the CONTROL register is read first. Afterward, the RXDONE bit is checked using an **andi** instruction (to mask the desired bit). As long as this bit is 0, it means no data has been received, and the value is read again (branching back to the start of the check). When RXDONE is 1, the loop breaks, and a zero is written to CONTROL to clear the RXDONE bit. Immediately after, the value of RXDATA is read.

The process to send data is similar: first, the value of the CONTROL register is read. Then, the TXDONE bit is checked using the **andi** instruction. As long as this bit is 0, it means the transmitter is currently transferring a value and we must wait. Thus, the value is read again (branching back to the check). When TXDONE is 1, the loop breaks, and the value is written to TXDATA.


ChampionCHIP
eXperience